

Name M Ayyan Nasir

SAP ID 57068

Assignment no 3

About Dota 2 Game:

1. Game Overview

Selected Game: Dota 2

Dota 2 is a Multiplayer Online Battle Arena (MOBA) game where two teams of five players compete to destroy the enemy Ancient. Each player controls a unique hero with specialized abilities and works with teammates to gain map control, defeat opponents, and achieve objectives.

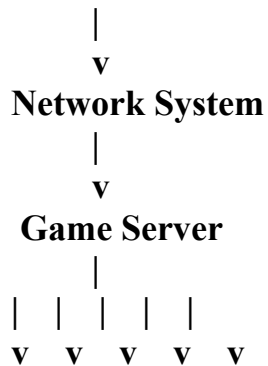
Core Gameplay Mechanics

- Hero selection
- Team-based combat
- Ability casting
- Gold and experience progression
- Item purchasing
- Matchmaking
- Real-time multiplayer networking
- Victory condition through Ancient destruction

2. High-Level Architecture

- Major Systems

Client Layer



Player System Combat System Inventory System

Matchmaking System

Game State Manager

UI System

Player System

Responsibilities:

- Player authentication
- Hero ownership
- Statistics tracking
- Match history

Matchmaking System

Responsibilities:

- Queue management

- Skill rating calculation
- Team balancing
- Match creation

Combat System

Responsibilities:

- Damage calculations
- Ability execution
- Cooldowns
- Status effects
- Death and respawn logic

Inventory System

Responsibilities:

- Item storage
- Item purchases
- Equipment management
- Item effects

Game State Management

Responsibilities:

- Match lifecycle

- Timer management
- Objective tracking
- Win condition detection

UI System

Responsibilities:

- HUD display
- Inventory interface
- Scoreboard
- Minimap

3. Code Structure

- **Project Structure**

1. src/

- **Core/**
GameManager
EventSystem
NetworkManager
- **Players/**
Player
Hero
HeroStats
- **Combat/**
Ability
DamageCalculator
EffectManager
CooldownManager

- **Inventory/
Item
Inventory
ShopSystem**
- **Matchmaking/
QueueManager
MatchCreator**
- **UI/

HUD
Minimap
Scoreboard**
- **Networking/
PacketHandler
ServerConnection**

4. Design Decisions

Component-Based Architecture

Reason:

- Promotes modularity.
- Easier maintenance.
- Supports future expansion.

Server Authoritative Model

Reason:

- Prevents cheating.
- Maintains synchronization.
- Common industry standard for multiplayer games.

Event-Driven Communication

Reason:

- Reduces coupling.
- Improves scalability.
- Simplifies system interaction.